SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Code of Conduct:

I Vaishnavee Bhavani Vijayakumar (192571071) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
Vaishnavee Bhavani Vijayakumar

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.

(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.
(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Responsive Layout Design

(a) Analysis of Possible Reasons for Lack of Responsiveness

The common causes of layout failure across mobile and tablet viewports include:

- **Missing Viewport Meta Tag:** Omitting <meta name="viewport" content="width=device-width, initial-scale=1.0"> causes mobile browsers to render the page as a desktop canvas, forcing awkward zooming and panning.

- **Fixed Dimensions:** Using hardcoded pixel values (e.g., width: 1200px) instead of fluid units (%, vw, rem, fr) creates horizontal scrollbars and layout clipping.

- **Over-reliance on Floats and Absolute Positioning:** Floating elements without clearings or using fixed absolute offsets breaks the normal document flow on varying aspect ratios.

- **Unconstrained Media Assets:** Images and videos lacking max-width: 100% overflow their parent wrappers on narrow viewports.

(b) Responsive Webpage Layout Design (HTML & CSS)

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Responsive Retail Store</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="site-container">
    <header class="header">
      <div class="logo">RetailStore</div>
      <nav class="nav">
        <a href="#">Home</a>
        <a href="#">Shop</a>
```

```
        <a href="#">Deals</a>
        <a href="#">Contact</a>
      </nav>
    </header>
    <aside class="sidebar">
      <h3>Categories</h3>
      <ul>
        <li>Apparel</li>
        <li>Electronics</li>
        <li>Home & Living</li>
      </ul>
    </aside>
    <main class="content">
      <div class="product-grid">
        <article class="card">Product 1</article>
        <article class="card">Product 2</article>
        <article class="card">Product 3</article>
        <article class="card">Product 4</article>
      </div>
    </main>
    <footer class="footer">
      <p>&copy; 2026 RetailStore. All rights reserved.</p>
    </footer>
  </div>
</body>
</html>
```

(c) Multi-Device Adaptation Using CSS Grid & Media Queries

CSS

```
/* Base Mobile-First Styles (Default: Mobile < 768px) */
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
```

```
}
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  line-height: 1.5;
  color: #333;
}
.site-container {
  display: grid;
  grid-template-columns: 1fr;
  grid-template-areas:
    "header"
    "sidebar"
    "content"
    "footer";
  min-height: 100vh;
}
.header {
  grid-area: header;
  background-color: #1a202c;
  color: white;
  padding: 1rem;
  display: flex;
  flex-direction: column;
  gap: 0.75rem;
}
.nav {
  display: flex;
  flex-direction: column;
  gap: 0.5rem;
}
.nav a {
  color: #edf2f7;
  text-decoration: none;
}
```

```
.sidebar {
  grid-area: sidebar;
  background-color: #f7fafc;
  padding: 1rem;
}
.content {
  grid-area: content;
  padding: 1rem;
}
.product-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}
.card {
  background: #edf2f7;
  padding: 2rem;
  border-radius: 8px;
  text-align: center;
}
.footer {
  grid-area: footer;
  background-color: #1a202c;
  color: white;
  text-align: center;
  padding: 1rem;
}
/* Tablet Breakpoint (768px to 1023px) */
@media (min-width: 768px) {
  .header {
    flex-direction: row;
    justify-content: space-between;
    align-items: center;
  }
```

```
  .nav {
    flex-direction: row;
    gap: 1.5rem;
  }
  .site-container {
    grid-template-columns: 200px 1fr;
    grid-template-areas:
      "header  header"
      "sidebar content"
      "footer  footer";
  }
  .product-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}
/* Desktop Breakpoint (1024px and above) */
@media (min-width: 1024px) {
  .site-container {
    grid-template-columns: 260px 1fr;
    max-width: 1280px;
    margin: 0 auto;
  }
  .product-grid {
    grid-template-columns: repeat(4, 1fr);
    gap: 1.5rem;
  }
}
```

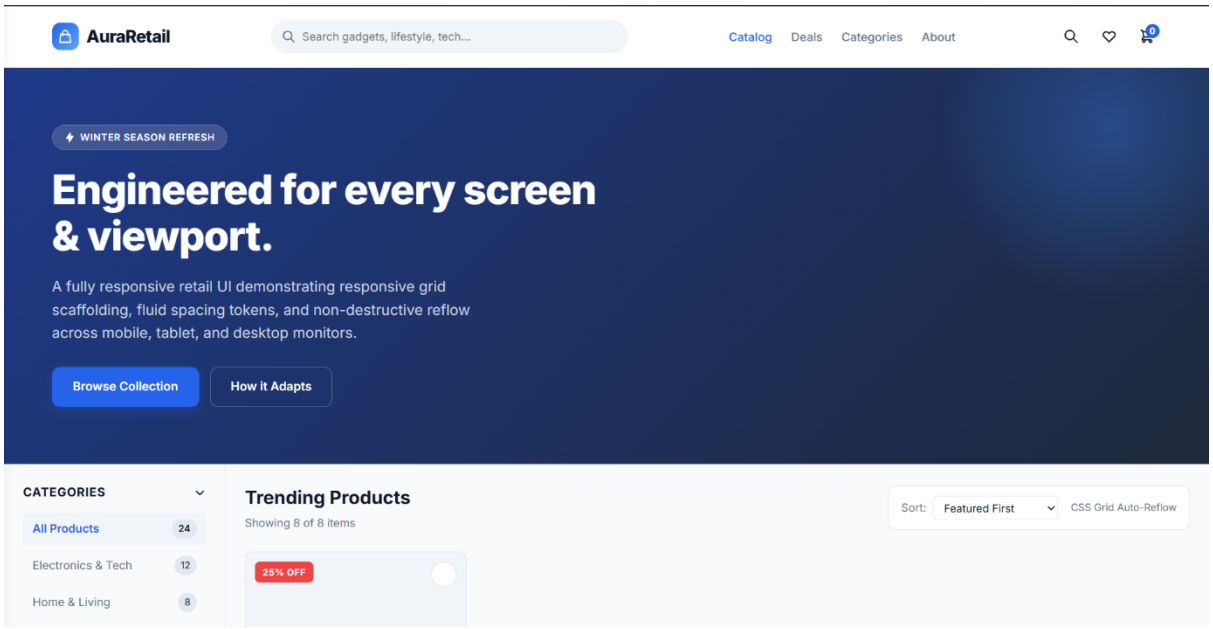
(d) Spacing, Alignment, and Distribution Techniques

- **Explicit Gap Management:** Replacing manual child margins with CSS gap eliminates margin-collapsing bugs across flex and grid containers.
- **Auto-Margins & Flex Centering:** Using margin: 0 auto; enforces consistent horizontal centering on wider displays, while align-items: center and justify-content: space-between manage clean internal alignments.

- **Fluid Spacing Tokens:** Applying relative unit scales (rem and clamp()) prevents content clipping across fluid viewports.

(e) Evaluation & Further Improvements

- **Evaluation:** CSS Grid (grid-template-areas) cleanly handles macro-layout restructuring, while Flexbox governs linear micro-components (headers, lists). Content adjusts dynamically without horizontal overflow.
- **Future Enhancements:**
    1. Implement clamp() for fluid typography (e.g., font-size: clamp(1rem, 2.5vw, 1.5rem)).
    2. Implement responsive images with <picture> and srcset to serve optimized resolutions per device.
    3. Integrate container queries (@container) so product cards adapt dynamically based on component size rather than full screen width.



Experiment 2: Form Validation using JavaScript

(a) Analysis of Issues in Existing Form Validation

- **Absence of Client-Side Checks:** Submitting unverified forms relies entirely on round-trips to the server, introducing latency, server overhead, and poor UX.
- **Loose or Non-Standard Validation:** Simple substring checks (e.g., verifying whether @ exists) fail on inputs like user@.com or user@domain.
- **Inadequate Edge-Case Handling:** Failing to strip whitespace leads to false negatives.
- **Intrusive Feedback:** Relying on modal dialogs (alert()) interrupts user workflow rather than highlighting errors contextually.

(b) HTML Form Implementation

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>User Registration</title>
  <style>
    .form-group {
      margin-bottom: 1.25rem;
      position: relative;
    }
    label {
      display: block;
      margin-bottom: 0.25rem;
      font-weight: 600;
    }
    input {
      width: 100%;
      max-width: 380px;
      padding: 0.5rem;
      border: 1.5px solid #cbd5e0;
      border-radius: 4px;
```

```
      font-size: 1rem;
    }
    input.invalid {
      border-color: #e53e3e;
      background-color: #fff5f5;
    }
    input.valid {
      border-color: #38a169;
    }
    .error-msg {
      display: block;
      color: #e53e3e;
      font-size: 0.85rem;
      margin-top: 0.25rem;
      min-height: 1.2rem;
    }
    button {
      padding: 0.6rem 1.2rem;
      background-color: #3182ce;
      color: white;
      border: none;
      border-radius: 4px;
      cursor: pointer;
    }
  </style>
</head>
<body>
  <form id="registrationForm" novalidate>
    <div class="form-group">
      <label for="email">Email Address</label>
      <input type="email" id="email" name="email" required autocomplete="email"
placeholder="name@example.com">
      <span class="error-msg" id="emailError" role="alert" aria-live="polite"></span>
    </div>
```

```
    <div class="form-group">
      <label for="phone">Phone Number</label>
      <input type="tel" id="phone" name="phone" required autocomplete="tel"
placeholder="10-digit number">
      <span class="error-msg" id="phoneError" role="alert" aria-live="polite"></span>
    </div>
    <button type="submit">Register</button>
  </form>
  <script src="validation.js"></script>
</body>
</html>
```

(c) & (d) Regular Expressions and Real-Time Inline Validation

JavaScript

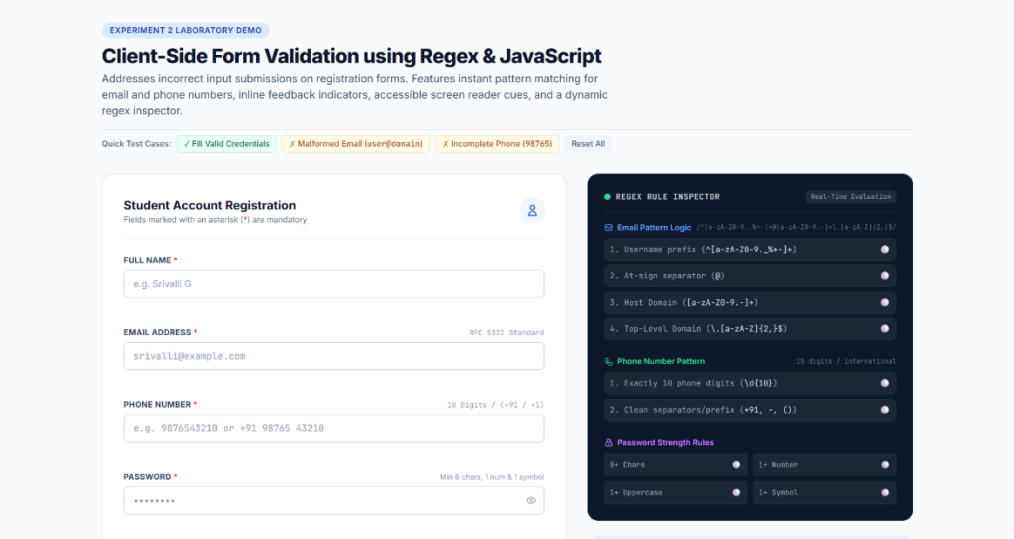
```
// validation.js
document.addEventListener("DOMContentLoaded", () => {
  const form = document.getElementById("registrationForm");
  const emailInput = document.getElementById("email");
  const phoneInput = document.getElementById("phone");
  const emailError = document.getElementById("emailError");
  const phoneError = document.getElementById("phoneError");
  // RFC 5322-compliant simplified regex pattern
  const emailRegex = /^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
  // Validates standard 10-digit formats: 1234567890, 123-456-7890, (123) 456-7890
  const phoneRegex = /^(\+?\d{1,3}[- ]?)?\(?\d{3}\)?[- ]?\d{3}[- ]?\d{4}$/;
  function validateField(input, regex, errorElement, emptyMsg, invalidMsg) {
    const value = input.value.trim();
    if (value === "") {
      showError(input, errorElement, emptyMsg);
      return false;
    }
    if (!regex.test(value)) {
      showError(input, errorElement, invalidMsg);
      return false;
```

```
    }
    showSuccess(input, errorElement);
    return true;
  }
  function showError(input, element, message) {
    input.classList.add("invalid");
    input.classList.remove("valid");
    element.textContent = message;
  }
  function showSuccess(input, element) {
    input.classList.remove("invalid");
    input.classList.add("valid");
    element.textContent = "";
  }
  // Real-time listener: 'input' triggers immediate inline feedback
  emailInput.addEventListener("input", () => {
    validateField(emailInput, emailRegex, emailError, "Email is required.", "Enter a
valid email address (e.g., user@domain.com).");
  });
  phoneInput.addEventListener("input", () => {
    validateField(phoneInput, phoneRegex, phoneError, "Phone number is required.",
"Enter a valid 10-digit phone number.");
  });
  // Submission handler
  form.addEventListener("submit", (e) => {
    const isEmailValid = validateField(emailInput, emailRegex, emailError, "Email is
required.", "Enter a valid email address.");
    const isPhoneValid = validateField(phoneInput, phoneRegex, phoneError, "Phone
number is required.", "Enter a valid 10-digit phone number.");
    if (!isEmailValid || !isPhoneValid) {
      e.preventDefault();
    } else {
      e.preventDefault();
      alert("Registration submitted successfully!");
```

```
      form.reset();
      emailInput.classList.remove("valid");
      phoneInput.classList.remove("valid");
    }
  });
});
```

(e) Evaluation and System Enhancements

- **UX Improvements:** The real-time input validation provides fast, contextual feedback without disruptive blocking dialogs. Using aria-live="polite" makes errors accessible to assistive screen readers.
- **Security & Defense-in-Depth:**
  - **Client-side is UX only:** Client-side validation can be bypassed by disabling JavaScript or calling endpoints directly via tools like cURL/Postman. Server-side validation with parameterized inputs is mandatory.
  - **Input Sanitization:** Sanitize data to prevent Cross-Site Scripting (XSS) and SQL Injection attacks prior to database persistence.



Experiment 3: CSS Layout Debugging

(a) CSS Properties Responsible for Layout Overlaps

- position: absolute / fixed: Takes elements out of normal document flow. If parent wrappers do not adjust height, sibling elements render directly beneath or over them.

- z-index: Misconfigured stacking contexts cause lower-priority elements to overlap text or navigation bars.

- overflow: visible with fixed heights: Setting fixed heights (e.g., height: 250px) causes dynamic text to spill into neighboring rows during resizing.

- float without clearfix: Sibling elements collapse upward behind floated blocks.

(b) Role of the CSS Box Model

The standard box model calculates element footprint as:

$$\text{Total Width} = \text{width} + \text{padding-left} + \text{padding-right} + \text{border-left} + \text{border-right} + \text{margin-left} + \text{margin-right}$$

Under default box-sizing: content-box, adding padding or borders expands the computed width beyond declared percentages (e.g., 50% + 50% + 20px padding > 100%), breaking multi-column layouts into overlapping or stacked blocks. Switching globally to box-sizing: border-box ensures padding and borders are calculated *inside* the declared width.

(c) & (d) Debugged CSS Implementation

Problematic CSS (Causes Overlap)

CSS

```
/* BROKEN LAYOUT */
.card-container {
  height: 300px; /* Fixed height causes content blowout */
}
.card {
  position: absolute; /* Removed from flow; cards overlap each other */
  width: 50%;
  left: 0;
}
```

Corrected CSS (Flow Maintained with Flexbox/Grid)

CSS

```
/* Reset with Border-Box */
*, *::before, *::after {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}
/* Parent Container */
.cards-wrapper {
  display: flex;
  flex-wrap: wrap; /* Allows wrapping instead of horizontal clumping */
  gap: 1.5rem;
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
  padding: 1rem;
}
/* Individual Cards */
.card {
  position: relative; /* Kept in normal document flow */
  flex: 1 1 calc(33.333% - 1.5rem); /* Responsive basis with gap allowance */
  min-width: 280px; /* Prevents extreme compression */
  min-height: 200px; /* Expands dynamically if text wraps */
  background: #ffffff;
  border: 1px solid #e2e8f0;
  border-radius: 8px;
  padding: 1.25rem;
  box-shadow: 0 2px 4px rgba(0,0,0,0.05);
}
```

(e) Evaluation and Best Practices for Visual Consistency

- **Preserve Document Flow:** Avoid position: absolute for page layout structuring; reserve absolute coordinates strictly for isolated UI accents (badges, tooltips, close buttons).

- **Avoid Hardcoded Heights:** Prefer min-height over height so containers stretch automatically if user fonts or screen dimensions change.

- **Adopt** border-box **Uniformly:** Apply * { box-sizing: border-box; } at the root of every CSS project.

- **Use Automated Linting and Browser DevTools:** Inspect computed styles in Chrome/Firefox DevTools and use CSS linters (e.g., Stylelint) to catch conflicting properties before deploying to production.

